

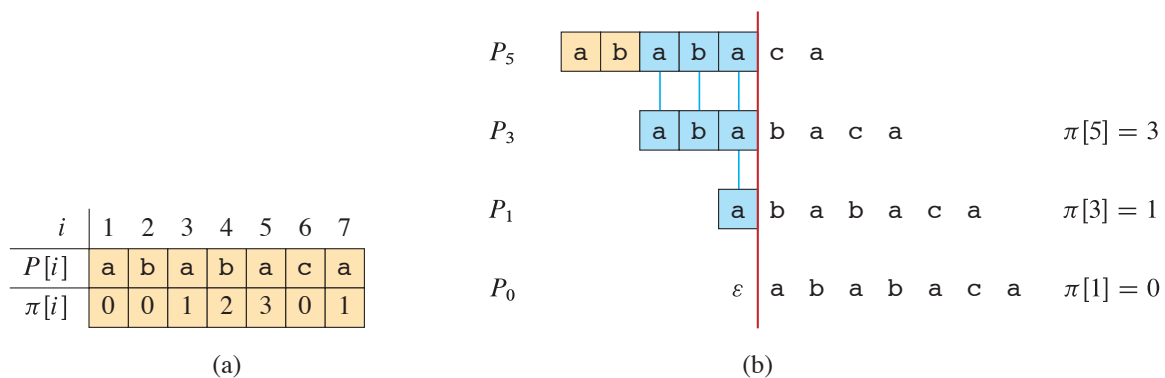


Figure 32.10 An illustration of Lemma 32.5 for the pattern $P = ababaca$ and $q = 5$. (a) The π function for the given pattern. Since $\pi[5] = 3$, $\pi[3] = 1$, and $\pi[1] = 0$, iterating π gives $\pi^*[5] = \{3, 1, 0\}$. (b) Sliding the template containing the pattern P to the right and noting when some prefix $P[:k]$ of P matches up with some proper suffix of $P[:5]$. Matches occur when $k = 3, 1$, and 0 . In the figure, the first row gives P , and the vertical red line is drawn just after $P[:5]$. Successive rows show all the shifts of P that cause some prefix $P[:k]$ of P to match some suffix of $P[:5]$. Successfully matched characters are shown in blue. Blue lines connect aligned matching characters. Thus, $\{k : k < 5 \text{ and } P[:k] \sqsupseteq P[:5]\} = \{3, 1, 0\}$. Lemma 32.5 claims that $\pi^*[q] = \{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}$ for all q .

Compared with FINITE-AUTOMATON-MATCHER, by using π rather than δ , the KMP algorithm reduces the time for preprocessing the pattern from $O(m |\Sigma|)$ to $\Theta(m)$, while keeping the actual matching time bounded by $\Theta(n)$.

Correctness of the prefix-function computation

We'll see a little later that the prefix function π helps to simulate the transition function δ in a string-matching automaton. But first, we need to prove that the procedure COMPUTE-PREFIX-FUNCTION does indeed compute the prefix function correctly. Doing so requires finding all prefixes $P[:k]$ that are proper suffixes of a given prefix $P[:q]$. The value of $\pi[q]$ gives us the length of the longest such prefix, but the following lemma, illustrated in Figure 32.10, shows that iterating the prefix function π generates all the prefixes $P[:k]$ that are proper suffixes of $P[:q]$. Let

$$\pi^*[q] = \{\pi[q], \pi^{(2)}[q], \pi^{(3)}[q], \dots, \pi^{(t)}[q]\} ,$$

where $\pi^{(i)}[q]$ is defined in terms of functional iteration, so that $\pi^{(0)}[q] = q$ and $\pi^{(i)}[q] = \pi[\pi^{(i-1)}[q]]$ for $i \geq 1$ (so that $\pi[q] = \pi^{(1)}[q]$), and where the sequence in $\pi^*[q]$ stops upon reaching $\pi^{(t)}[q] = 0$ for some $t \geq 1$.

Lemma 32.5 (Prefix-function iteration lemma)

Let P be a pattern of length m with prefix function π . Then, for $q = 1, 2, \dots, m$, we have $\pi^*[q] = \{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}$.

Proof We first prove that $\pi^*[q] \subseteq \{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}$ or, equivalently,

$$i \in \pi^*[q] \text{ implies } P[:i] \sqsupseteq P[:q]. \quad (32.7)$$

If $i \in \pi^*[q]$, then $i = \pi^{(u)}[q]$ for some $u > 0$. We prove equation (32.7) by induction on u . For $u = 1$, we have $i = \pi[q]$, and the claim follows since $i < q$ and $P[:\pi[q]] \sqsupseteq P[:q]$ by the definition of π . Now consider some $u \geq 1$ such that both $\pi^{(u)}[q]$ and $\pi^{(u+1)}[q]$ belong to $\pi^*[q]$. Let $i = \pi^{(u)}[q]$, so that $\pi[i] = \pi^{(u+1)}[q]$. The inductive hypothesis is that $P[:i] \sqsupseteq P[:q]$. Because the relations $<$ and $\sqsupseteq$ are transitive, we have $\pi[i] < i < q$ and $P[:\pi[i]] \sqsupseteq P[:i] \sqsupseteq P[:q]$, which establishes equation (32.7) for all i in $\pi^*[q]$. Therefore, $\pi^*[q] \subseteq \{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}$.

We now prove that $\{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\} \subseteq \pi^*[q]$ by contradiction. Suppose to the contrary that the set $\{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\} - \pi^*[q]$ is nonempty, and let j be the largest number in the set. Because $\pi[q]$ is the largest value in $\{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}$ and $\pi[q] \in \pi^*[q]$, it must be the case that $j < \pi[q]$. Having established that $\pi^*[q]$ contains at least one integer greater than j , let j' denote the smallest such integer. (We can choose $j' = \pi[q]$ if no other number in $\pi^*[q]$ is greater than j .) We have $P[:j] \sqsupseteq P[:q]$ because $j \in \{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}$, and from $j' \in \pi^*[q]$ and equation (32.7), we have $P[:j'] \sqsupseteq P[:q]$. Thus, $P[:j] \sqsupseteq P[:j']$ by Lemma 32.1, and j is the largest value less than j' with this property. Therefore, we must have $\pi[j'] = j$ and, since $j' \in \pi^*[q]$, we must have $j \in \pi^*[q]$ as well. This contradiction proves the lemma. ■

The algorithm COMPUTE-PREFIX-FUNCTION computes $\pi[q]$, in order, for $q = 1, 2, \dots, m$. Setting $\pi[1]$ to 0 in line 2 of COMPUTE-PREFIX-FUNCTION is certainly correct, since $\pi[q] < q$ for all q . We'll use the following lemma and its corollary to prove that COMPUTE-PREFIX-FUNCTION computes $\pi[q]$ correctly for $q > 1$.

Lemma 32.6

Let P be a pattern of length m , and let π be the prefix function for P . For $q = 1, 2, \dots, m$, if $\pi[q] > 0$, then $\pi[q] - 1 \in \pi^*[q - 1]$.

Proof Let $r = \pi[q] > 0$, so that $r < q$ and $P[:r] \sqsupseteq P[:q]$, and thus, $r - 1 < q - 1$ and $P[:r - 1] \sqsupseteq P[:q - 1]$ (by dropping the last character from

$P[:r]$ and $P[:q]$, which we can do because $r > 0$). By Lemma 32.5, therefore, $r - 1 \in \pi^*[q - 1]$. Thus, we have $\pi[q] - 1 = r - 1 \in \pi^*[q - 1]$. ■

For $q = 2, 3, \dots, m$, define the subset $E_{q-1} \subseteq \pi^*[q - 1]$ by

$$\begin{aligned} E_{q-1} &= \{k \in \pi^*[q - 1] : P[k + 1] = P[q]\} \\ &= \{k : k < q - 1 \text{ and } P[:k] \sqsupset P[:q - 1] \text{ and } P[k + 1] = P[q]\} \\ &\quad \text{(by Lemma 32.5)} \\ &= \{k : k < q - 1 \text{ and } P[:k + 1] \sqsupset P[:q]\} . \end{aligned}$$

The set E_{q-1} consists of the values $k < q - 1$ for which $P[:k] \sqsupset P[:q - 1]$ and for which, because $P[k + 1] = P[q]$, we have $P[:k + 1] \sqsupset P[:q]$. Thus, E_{q-1} consists of those values $k \in \pi^*[q - 1]$ such that extending $P[:k]$ to $P[:k + 1]$ produces a proper suffix of $P[:q]$.

Corollary 32.7

Let P be a pattern of length m , and let π be the prefix function for P . Then, for $q = 2, 3, \dots, m$,

$$\pi[q] = \begin{cases} 0 & \text{if } E_{q-1} = \emptyset , \\ 1 + \max E_{q-1} & \text{if } E_{q-1} \neq \emptyset . \end{cases}$$

Proof If E_{q-1} is empty, there is no $k \in \pi^*[q - 1]$ (including $k = 0$) such that extending $P[:k]$ to $P[:k + 1]$ produces a proper suffix of $P[:q]$. Therefore, $\pi[q] = 0$.

If, instead, E_{q-1} is nonempty, then for each $k \in E_{q-1}$, we have $k + 1 < q$ and $P[:k + 1] \sqsupset P[:q]$. Therefore, the definition of $\pi[q]$ gives

$$\pi[q] \geq 1 + \max E_{q-1} . \tag{32.8}$$

Note that $\pi[q] > 0$. Let $r = \pi[q] - 1$, so that $r + 1 = \pi[q] > 0$, and therefore $P[:r + 1] \sqsupset P[:q]$. If a nonempty string is a suffix of another, then the two strings must have the same last character. Since $r + 1 > 0$, the prefix $P[:r + 1]$ is nonempty, and so $P[r + 1] = P[q]$. Furthermore, $r \in \pi^*[q - 1]$ by Lemma 32.6. Therefore, $r \in E_{q-1}$, and so $\pi[q] - 1 = r \leq \max E_{q-1}$ or, equivalently,

$$\pi[q] \leq 1 + \max E_{q-1} . \tag{32.9}$$

Combining equations (32.8) and (32.9) completes the proof. ■

We now finish the proof that COMPUTE-PREFIX-FUNCTION computes π correctly. The key is to combine the definition of E_{q-1} with the statement of Corollary 32.7, so that $\pi[q]$ equals 1 plus the greatest value of k in $\pi^*[q - 1]$ such that

$P[k + 1] = P[q]$. First, in COMPUTE-PREFIX-FUNCTION, $k = \pi[q - 1]$ at the start of each iteration of the **for** loop of lines 4–9. This condition is enforced by lines 2 and 3 when the loop is first entered, and it remains true in each successive iteration because of line 9. Lines 5–8 adjust k so that it becomes the correct value of $\pi[q]$. The **while** loop of lines 5–6 searches through all values $k \in \pi^*[q - 1]$ in decreasing order to find the value of $\pi[q]$. The loop terminates either because k reaches 0 or $P[k + 1] = P[q]$. Because the “and” operator short-circuits, if the loop terminates because $P[k + 1] = P[q]$, then k must have also been positive, and so k is the greatest value in E_{q-1} . In this case, lines 7–9 set $\pi[q]$ to $k + 1$, according to Corollary 32.7. If, instead, the **while** loop terminates because $k = 0$, then there are two possibilities. If $P[1] = P[q]$, then $E_{q-1} = \{0\}$, and lines 7–9 set both k and $\pi[q]$ to 1. If $k = 0$ and $P[1] \neq P[q]$, however, then $E_{q-1} = \emptyset$. In this case, line 9 sets $\pi[q]$ to 0, again according to Corollary 32.7, which completes the proof of the correctness of COMPUTE-PREFIX-FUNCTION.

Correctness of the Knuth-Morris-Pratt algorithm

You can think of the procedure KMP-MATCHER as a reimplemented version of the procedure FINITE-AUTOMATON-MATCHER, but using the prefix function π to compute state transitions. Specifically, we’ll prove that in the i th iteration of the **for** loops of both KMP-MATCHER and FINITE-AUTOMATON-MATCHER, the state q has the same value upon testing for equality with m (at line 8 in KMP-MATCHER and at line 4 in FINITE-AUTOMATON-MATCHER). Once we have argued that KMP-MATCHER simulates the behavior of FINITE-AUTOMATON-MATCHER, the correctness of KMP-MATCHER follows from the correctness of FINITE-AUTOMATON-MATCHER (though we’ll see a little later why line 10 in KMP-MATCHER is necessary).

Before formally proving that KMP-MATCHER correctly simulates FINITE-AUTOMATON-MATCHER, let’s take a moment to understand how the prefix function π replaces the δ transition function. Recall that when a string-matching automaton is in state q and it scans a character $a = T[i]$, it moves to a new state $\delta(q, a)$. If $a = P[q + 1]$, so that a continues to match the pattern, then the state number is incremented: $\delta(q, a) = q + 1$. Otherwise, $a \neq P[q + 1]$, so that a does not continue to match the pattern, and the state number does not increase: $0 \leq \delta(q, a) \leq q$. In the first case, when a continues to match, KMP-MATCHER moves to state $q + 1$ without referring to the π function: the **while** loop test in line 4 immediately comes up false, the test in line 6 comes up true, and line 7 increments q .

The π function comes into play when the character a does not continue to match the pattern, so that the new state $\delta(q, a)$ is either q or to the left of q along the spine of the automaton. The **while** loop of lines 4–5 in KMP-MATCHER iterates through

the states in $\pi^*[q]$, stopping either when it arrives in a state, say q' , such that a matches $P[q' + 1]$ or q' has gone all the way down to 0. If a matches $P[q' + 1]$, then line 7 sets the new state to $q' + 1$, which should equal $\delta(q, a)$ for the simulation to work correctly. In other words, the new state $\delta(q, a)$ should be either state 0 or a state numbered 1 more than some state in $\pi^*[q]$.

Let's look at the example in Figures 32.6 and 32.10, which are for the pattern $P = \text{ababaca}$. Suppose that the automaton is in state $q = 5$, having matched ababa . The states in $\pi^*[5]$ are, in descending order, 3, 1, and 0. If the next character scanned is c , then you can see that the automaton moves to state $\delta(5, \text{c}) = 6$ in both `FINITE-AUTOMATON-MATCHER` (line 3) and `KMP-MATCHER` (line 7). Now suppose that the next character scanned is instead b , so that the automaton should move to state $\delta(5, \text{b}) = 4$. The **while** loop in `KMP-MATCHER` exits after executing line 5 once, and the automaton arrives in state $q' = \pi[5] = 3$. Since $P[q' + 1] = P[4] = \text{b}$, the test in line 6 comes up true, and the automaton moves to the new state $q' + 1 = 4 = \delta(5, \text{b})$. Finally, suppose that the next character scanned is instead a , so that the automaton should move to state $\delta(5, \text{a}) = 1$. The first three times that the test in line 4 executes, the test comes up true. The first time finds that $P[6] = \text{c} \neq \text{a}$, and the automaton moves to state $\pi[5] = 3$ (the first state in $\pi^*[5]$). The second time finds that $P[4] = \text{b} \neq \text{a}$, and the automaton moves to state $\pi[3] = 1$ (the second state in $\pi^*[5]$). The third time finds that $P[2] = \text{b} \neq \text{a}$, and the automaton moves to state $\pi[1] = 0$ (the last state in $\pi^*[5]$). The **while** loop exits once it arrives in state $q' = 0$. Now line 6 finds that $P[q' + 1] = P[1] = \text{a}$, and line 7 moves the automaton to the new state $q' + 1 = 1 = \delta(5, \text{a})$.

Thus, the intuition is that `KMP-MATCHER` iterates through the states in $\pi^*[q]$ in decreasing order, stopping at some state q' and then possibly moving to state $q' + 1$. Although that might seem like a lot of work just to simulate computing $\delta(q, a)$, bear in mind that asymptotically, `KMP-MATCHER` is no slower than `FINITE-AUTOMATON-MATCHER`.

We are now ready to formally prove the correctness of the Knuth-Morris-Pratt algorithm. By Theorem 32.4, we have that $q = \sigma(T[:i])$ after each time line 3 of `FINITE-AUTOMATON-MATCHER` executes. Therefore, it suffices to show that the same property holds with regard to the **for** loop in `KMP-MATCHER`. The proof proceeds by induction on the number of loop iterations. Initially, both procedures set q to 0 as they enter their respective **for** loops for the first time. Consider iteration i of the **for** loop in `KMP-MATCHER`. By the inductive hypothesis, the state number q equals $\sigma(T[:i - 1])$ at the start of the loop iteration. We need to show that when line 8 is reached, the new value of q is $\sigma(T[:i])$. (Again, we'll handle line 10 separately.)

Considering q to be the state number at the start of the **for** loop iteration, when `KMP-MATCHER` considers the character $T[i]$, the longest prefix of P that is a suffix of $T[:i]$ is either $P[:q + 1]$ (if $P[q + 1] = T[i]$) or some prefix (not

necessarily proper, and possibly empty) of $P[:q]$. We consider separately the three cases in which $\sigma(T[:i]) = 0$, $\sigma(T[:i]) = q + 1$, and $0 < \sigma(T[:i]) \leq q$.

- If $\sigma(T[:i]) = 0$, then $P[:0] = \varepsilon$ is the only prefix of P that is a suffix of $T[:i]$. The **while** loop of lines 4–5 iterates through each value q' in $\pi^*[q]$, but although $P[:q'] \sqsupset P[:q] \sqsupset T[:i-1]$ for every $q' \in \pi^*[q]$ (because $<$ are $\sqsupset$ are transitive relations), the loop never finds a q' such that $P[q'+1] = T[i]$. The loop terminates when q reaches 0, and of course line 7 does not execute. Therefore, $q = 0$ at line 8, so that now $q = \sigma(T[:i])$.
- If $\sigma(T[:i]) = q + 1$, then $P[q+1] = T[i]$, and the **while** loop test in line 4 fails the first time through. Line 7 executes, incrementing the state number to $q + 1$, which equals $\sigma(T[:i])$.
- If $0 < \sigma(T[:i]) \leq q'$, then the **while** loop of lines 4–5 iterates at least once, checking in decreasing order each value in $\pi^*[q]$ until it stops at some $q' < q$. Thus, $P[:q']$ is the longest prefix of $P[:q]$ for which $P[q'+1] = T[i]$, so that when the **while** loop terminates, $q' + 1 = \sigma(P[:q]T[i])$. Since $q = \sigma(T[:i-1])$, Lemma 32.3 implies that $\sigma(T[:i-1]T[i]) = \sigma(P[:q]T[i])$. Thus we have

$$\begin{aligned} q' + 1 &= \sigma(P[:q]T[i]) \\ &= \sigma(T[:i-1]T[i]) \\ &= \sigma(T[:i]) \end{aligned}$$

when the **while** loop terminates. After line 7 increments q , the new state number q equals $\sigma(T[:i])$.

Line 10 is necessary in KMP-MATCHER, because otherwise, line 4 might try to reference $P[m+1]$ after finding an occurrence of P . (The argument that $q = \sigma(T[:i-1])$ upon the next execution of line 4 remains valid by the hint given in Exercise 32.4-8: that $\delta(m, a) = \delta(\pi[m]a)$ or, equivalently, $\sigma(Pa) = \sigma(P[:\pi[m]]a)$ for any $a \in \Sigma$.) The remaining argument for the correctness of the Knuth-Morris-Pratt algorithm follows from the correctness of FINITE-AUTOMATON-MATCHER, since we have shown that KMP-MATCHER simulates the behavior of FINITE-AUTOMATON-MATCHER.

Exercises

32.4-1

Compute the prefix function π for the pattern ababbabbabbababbabb.

32.4-2

Give an upper bound on the size of $\pi^*[q]$ as a function of q . Give an example to show that your bound is tight.

32.4-3

Explain how to determine the occurrences of pattern P in the text T by examining the π function for the string PT (the string of length $m+n$ that is the concatenation of P and T).

32.4-4

Use an aggregate analysis to show that the running time of KMP-MATCHER is $\Theta(n)$.

32.4-5

Use a potential function to show that the running time of KMP-MATCHER is $\Theta(n)$.

32.4-6

Show how to improve KMP-MATCHER by replacing the occurrence of π in line 5 (but not line 10) by π' , where π' is defined recursively for $q = 1, 2, \dots, m-1$ by the equation

$$\pi'[q] = \begin{cases} 0 & \text{if } \pi[q] = 0, \\ \pi'[\pi[q]] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] = P[q + 1], \\ \pi[q] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] \neq P[q + 1]. \end{cases}$$

Explain why the modified algorithm is correct, and explain in what sense this change constitutes an improvement.

32.4-7

Give a linear-time algorithm to determine whether a text T is a cyclic rotation of another string T' . For example, `braze` and `zebra` are cyclic rotations of each other.

★ 32.4-8

Give an $O(m|\Sigma|)$ -time algorithm for computing the transition function δ for the string-matching automaton corresponding to a given pattern P . (*Hint:* Prove that $\delta(q, a) = \delta(\pi[q]a)$ if $q = m$ or $P[q + 1] \neq a$.)

32.5 Suffix arrays

The algorithms we have seen thus far in this chapter can efficiently find all occurrences of a pattern in a text. That is, however, all they can do. This section presents a different approach—suffix arrays—with which you can find all occurrences of a pattern in a text, but also quite a bit more. A suffix array won't find all occurrences

i	1	2	3	4	5	6	7
$T[i]$	r	a	t	a	t	a	t

i	$SA[i]$	$rank[i]$	$LCP[i]$	suffix $T[SA[i]:]$
1	6	4	0	at
2	4	3	2	atat
3	2	7	4	atatat
4	1	2	0	ratatat
5	7	6	0	t
6	5	1	1	tat
7	3	5	3	tatat

Figure 32.11 The suffix array SA , rank array $rank$, longest common prefix array LCP , and lexicographically sorted suffixes of the text $T = \text{ratatat}$ with length $n = 7$. The value of $rank[i]$ indicates the position of the suffix $T[i:]$ in the lexicographically sorted order: $rank[SA[i]] = i$ for $i = 1, 2, \dots, n$. The $rank$ array is used to compute the LCP array.

of a pattern as quickly as, say, the Knuth-Morris-Pratt algorithm, but its additional flexibility makes it well worth studying.

A suffix array is simply a compact way to represent the lexicographically sorted order of all n suffixes of a length- n text. Given a text $T[1:n]$, let $T[i:]$ denote the suffix $T[i:n]$. The **suffix array** $SA[1:n]$ of T is defined such that if $SA[i] = j$, then $T[j:]$ is the i th suffix of T in lexicographic order.³ That is, the i th suffix of T in lexicographic order is $T[SA[i]:]$. Along with the suffix array, another useful array is the **longest common prefix array** $LCP[1:n]$. The entry $LCP[i]$ gives the length of the longest common prefix between the i th and $(i - 1)$ st suffixes in the sorted order (with $LCP[SA[1]]$ defined to be 0, since there is no prefix lexicographically smaller than $T[SA[1]:]$). Figure 32.11 shows the suffix array and longest common prefix array for the 7-character text `ratatat`.

Given the suffix array for a text, you can search for a pattern via binary search on the suffix array. Each occurrence of a pattern in the text starts some suffix of the text, and because the suffix array is in lexicographically sorted order, all occurrences of a pattern will appear at the start of consecutive entries of the suffix array. For example, in Figure 32.11, the three occurrences of `at` in `ratatat` appear in entries 1 through 3 of the suffix array. If you find the length- m pattern in the length- n suffix array via binary search (taking $O(m \lg n)$ time because each comparison takes $O(m)$ time), then you can find all occurrences of the pattern in the text by searching backward and forward from that spot until you find a suffix that does not start with the pattern (or you go beyond the bounds of the suffix array). If the pattern occurs k times, then the time to find all k occurrences is $O(m \lg n + km)$.

³ Informally, lexicographic order is “alphabetical order” in the underlying character set. A more precise definition of lexicographic order appears in Problem 12-2 on page 327.

With the longest common prefix array, you can find a longest repeated substring, that is, the longest substring that occurs more than once in the text. If $LCP[i]$ contains a maximum value in the LCP array, then a longest repeated substring appears in $T[SA[i] : SA[i] + LCP[i] - 1]$. In the example of Figure 32.11, the LCP array has one maximum value: $LCP[3] = 4$. Therefore, since $SA[3] = 2$, the longest repeated substring is $T[2 : 5] = \text{atat}$. Exercise 32.5-3 asks you to use the suffix array and longest common prefix array to find the longest common substrings between two texts. Next, we'll see how to compute the suffix array for an n -character text in $O(n \lg n)$ time and, given the suffix array and the text, how to compute the longest common prefix array in $\Theta(n)$ time.

Computing the suffix array

There are several algorithms to compute the suffix array of a length- n text. Some run in linear time, but are rather complicated. One such algorithm is given in Problem 32-2. Here we'll explore a simpler algorithm that runs in $\Theta(n \lg n)$ time.

The idea behind the $O(n \lg n)$ -time procedure COMPUTE-SUFFIX-ARRAY on the following page is to lexicographically sort substrings of the text with increasing lengths. The procedure makes several passes over the text, with the substring length doubling each time. By the $\lceil \lg n \rceil$ th pass, the procedure is sorting all the suffixes, thereby gaining the information needed to construct the suffix array. The key to attaining an $O(n \lg n)$ -time algorithm will be to have each pass after the first sort in linear time, which will indeed be possible by using radix sort.

Let's start with a simple observation. Consider any two strings, s_1 and s_2 . Decompose s_1 into s'_1 and s''_1 , so that s_1 is s'_1 concatenated with s''_1 . Likewise, let s_2 be s'_2 concatenated with s''_2 . Now, suppose that s'_1 is lexicographically smaller than s'_2 . Then, regardless of s''_1 and s''_2 , it must be the case that s_1 is lexicographically smaller than s_2 . For example, let $s_1 = \text{aaz}$ and $s_2 = \text{aba}$, and decompose s_1 into $s'_1 = \text{aa}$ and $s''_1 = \text{z}$ and s_2 into $s'_2 = \text{ab}$ and $s''_2 = \text{a}$. Because s'_1 is lexicographically smaller than s'_2 , it follows that s_1 is lexicographically smaller than s_2 , even though s''_2 is lexicographically smaller than s''_1 .

Instead of comparing substrings directly, COMPUTE-SUFFIX-ARRAY represents substrings of the text with integer *ranks*. Ranks have the simple property that one substring is lexicographically smaller than another if and only if it has a smaller rank. Identical substrings have equal ranks.

Where do these ranks come from? Initially, the substrings being considered are just single characters from the text. Assume that, as in many programming languages, there is a function, `ord`, that maps a character to its underlying encoding, which is a positive integer. The `ord` function could be the ASCII or Unicode encodings or any other function that produces a relative ordering of the characters. For example if all the characters are known to be lowercase letters, then `ord(a) = 1`,

COMPUTE-SUFFIX-ARRAY(T, n)

```

1  allocate arrays substr-rank[1 :  $n$ ], rank[1 :  $n$ ], and SA[1 :  $n$ ]
2  for  $i = 1$  to  $n$ 
3      substr-rank[ $i$ ].left-rank = ord( $T[i]$ )
4      if  $i < n$ 
5          substr-rank[ $i$ ].right-rank = ord( $T[i + 1]$ )
6      else substr-rank[ $i$ ].right-rank = 0
7      substr-rank[ $i$ ].index =  $i$ 
8  sort the array substr-rank into monotonically increasing order based
    on the left-rank attributes, using the right-rank attributes to break ties;
    if still a tie, the order does not matter
9   $l = 2$ 
10 while  $l < n$ 
11     MAKE-RANKS(substr-rank, rank,  $n$ )
12     for  $i = 1$  to  $n$ 
13         substr-rank[ $i$ ].left-rank = rank[ $i$ ]
14         if  $i + l \leq n$ 
15             substr-rank[ $i$ ].right-rank = rank[ $i + l$ ]
16         else substr-rank[ $i$ ].right-rank = 0
17         substr-rank[ $i$ ].index =  $i$ 
18     sort the array substr-rank into monotonically increasing order based
        on the left-rank attributes, using the right-rank attributes
        to break ties; if still a tie, the order does not matter
19      $l = 2l$ 
20 for  $i = 1$  to  $n$ 
21     SA[ $i$ ] = substr-rank[ $i$ ].index
22 return SA

```

MAKE-RANKS(*substr-rank*, *rank*, n)

```

1   $r = 1$ 
2  rank[substr-rank[1].index] =  $r$ 
3  for  $i = 2$  to  $n$ 
4      if substr-rank[ $i$ ].left-rank  $\neq$  substr-rank[ $i - 1$ ].left-rank
        or substr-rank[ $i$ ].right-rank  $\neq$  substr-rank[ $i - 1$ ].right-rank
5           $r = r + 1$ 
6      rank[substr-rank[ $i$ ].index] =  $r$ 

```